

Lost & Found Platform

Day 1 — Complete Development & Learning Documentation

A beginner-friendly record of what was built, why each part was needed, how the pieces connect, and the key React / Express / MongoDB concepts learned.

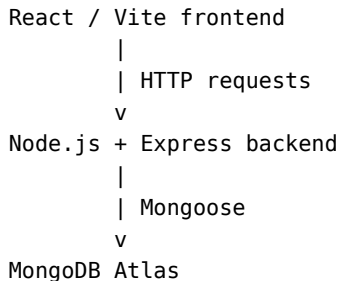
Area	Day 1 result
Frontend	React + Vite application with pages, components, routing and filters
Backend	Node.js + Express server with GET and POST item APIs
Database	MongoDB Atlas + Mongoose connection
Data model	Item schema containing name, category, description, date, location, image and status
Working flow	Report Lost form → Express POST API → MongoDB → GET API

Important assessment note: The assessment explicitly says AI tools must not be used to generate, modify, or complete the project. This document is therefore a study/reference record of the work already carried out, not a replacement for understanding the code.

1. Day 1 Goal

The goal of Day 1 was to establish the foundation of the Lost & Found web application. The application needs a frontend for users, a backend API, and a database for persistent item data.

The architecture built today



Think of the three layers like this: React is the screen the user interacts with; Express is the middleman that receives requests; MongoDB is where persistent data is stored.

2. Project Folder and Environment

The project was created under a Windows folder. An early problem occurred because the original folder name contained '&', which PowerShell interpreted specially. Renaming it to lost-and-found fixed the issue.

```
C:\Users\scl\Downloads\lost-and-found
├── client
└── server
```

Node/npm verification

```
node -v
npm -v
```

The environment used Node.js 24.11.1 and npm 11.6.2. The frontend was created with Vite.

3. Creating the React + Vite Frontend

```
npm create vite@latest client
```

During the Vite setup, React and JavaScript were selected. Vite supplied the initial React project structure and development server.

Why Vite?

- It gives a quick development server and a simple modern React project structure.
- The development server runs locally, such as `http://localhost:5173/`.
- It provides the build tooling needed to run the React application.

4. Understanding App.jsx

App.jsx became the main application component. Instead of keeping every piece of UI in one file, the project was split into components and pages.

```
import Navbar from "../components/Navbar"
import Home from "../pages/Home"
```

```
import './App.css'

function App() {
  return (
    <div>
      <Navbar />
      <Home />
    </div>
  )
}

export default App
```

The important idea is composition: App renders smaller components rather than containing the entire application itself.

5. Building the Navbar Component

A reusable Navbar component was created in `src/components/Navbar.jsx`. It contains the main navigation links, including Home, Browse, Report Item, Login and Sign Up.

Why make it a component?

- The navbar appears on multiple pages.
- A component avoids repeating the same navigation markup.
- Future changes can be made in one place.

6. CSS Foundation

The global CSS reset was placed in `index.css`, while application-specific styles were kept in `App.css`.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background-color: #ffffff;
  color: #222222;
}
```

`box-sizing: border-box` makes width and height calculations easier because padding and borders are included inside the declared dimensions.

7. Home Page

`Home.jsx` was created as the main landing page. It contains the hero section, report actions, and a browse section. Buttons were added for reporting lost/found items and browsing listings.

Important UI ideas learned

- Sections organize a page into meaningful areas.
- Buttons provide clear user actions.
- Flexbox was used to arrange cards.
- Hover styles improve feedback when the user points at an interactive element.

8. Reusable ItemCard Component and Props

Instead of writing the same card markup three times, `ItemCard.jsx` was created. It receives information through props.

```
function ItemCard(props) {
  return (
    <div className="item-card">
      <h3>{props.name}</h3>
      <p>Type: {props.type}</p>
      <p>Location: {props.location}</p>
    </div>
  )
}
```

```
}
```

```
export default ItemCard
```

Example of passing props

```
<ItemCard  
  name="Black Backpack"  
  type="Lost"  
  location="Library"  
>
```

A useful lesson from debugging was that JavaScript/JSX property names are case-sensitive. Passing Type is not the same as passing type when the component reads props.type.

The card layout initially had a CSS selector problem. DevTools was used to inspect the computed styles, and an inline flex style was temporarily used to verify that the layout itself was correct. This was a practical debugging lesson: isolate whether the problem is JSX structure, class naming, or CSS.

9. React Router and Multiple Pages

The router package was installed with:

```
npm install react-router-dom
```

Routes were then added for Home, Browse, Report Lost, Report Found, Login and Sign Up.

```
import { BrowserRouter, Routes, Route } from "react-router-dom"
```

```
<BrowserRouter>
  <Navbar />

  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/browse" element={<Browse />} />
    <Route path="/report/lost" element={<ReportLost />} />
    <Route path="/report/found" element={<ReportFound />} />
    <Route path="/login" element={<Login />} />
    <Route path="/signup" element={<SignUp />} />
  </Routes>
</BrowserRouter>
```

BrowserRouter enables client-side routing. Routes holds the route definitions, and Route maps a URL path to a React component.

10. Browse Page

Browse started with three sample items and was later upgraded to support search and filtering.

Sample item data

```
const items = [
  {
    name: "Black Backpack",
    type: "Lost",
    location: "Library",
    category: "Bags"
  },
  {
    name: "iphone 13",
    type: "Found",
    location: "Cafeteria",
    category: "Electronics"
  },
  {
    name: "Key Chain",
    type: "Lost",
    location: "Block A",
    category: "Keys"
  }
]
```

11. Search and Filter Logic

This was one of the more complicated frontend parts of Day 1. React state was used for the search text, lost/found type, category, and later the server-provided items.

```
const [search, setSearch] = useState("")
const [type, setType] = useState("All")
const [category, setCategory] = useState("All")
```

What useState means

useState gives a component a value that can change. The first value is the current state, and the second value is the function used to update it.

```
const [search, setSearch] = useState("")
```

search → current search text
setSearch → changes the search text

Controlled search input

```
<input  
  type="text"  
  placeholder="Search items..."  
  value={search}  
  onChange={(e) => setSearch(e.target.value)}  
>
```

value={search} means the input displays the current React state. onChange runs whenever the user types. e.target.value is the new text in the input, and setSearch stores it in state.

Filtering

```
const filteredItems = items.filter((item) =>  
  item.name.toLowerCase().includes(search.toLowerCase()) &&  
  (type === "All" || item.type === type) &&  
  (category === "All" || item.category === category)  
)
```

The three conditions are combined with &&, meaning an item must satisfy all active filters. The 'All' checks disable a filter when the user hasn't selected a specific value.

Category dropdown

```
<select  
  value={category}  
  onChange={(e) => setCategory(e.target.value)}  
>  
  <option value="All">All Categories</option>  
  <option value="Bags">Bags</option>  
  <option value="Electronics">Electronics</option>  
  <option value="Keys">Keys</option>  
</select>
```

12. Backend Setup — Node.js + Express

A separate server folder was created. Express was installed and used to create an API server on port 5000.

```
npm init -y
npm install express cors
```

Basic server idea

```
const express = require("express")
const app = express()

app.use(express.json())

app.get("/", (req, res) => {
  res.send("Lost & Found API is running")
})

app.listen(5000, () => {
  console.log("Server running on port 5000")
})
```

`express.json()` allows Express to read JSON request bodies. `app.get()` defines a GET endpoint. `app.listen()` starts the server.

13. First API Endpoint

A temporary GET endpoint was used to prove that the backend was responding correctly:

```
app.get("/api/items", (req, res) => {
  res.json([
    {
      name: "Black Backpack",
      type: "Lost",
      location: "Library"
    },
    {
      name: "iphone 13",
      type: "Found",
      location: "Cafeteria"
    }
  ])
})
```

Opening `http://localhost:5000/api/items` in Chrome returned JSON. This confirmed that the backend route worked.

14. MongoDB Atlas Setup

A separate MongoDB Atlas project named Lost & Found was created so the Lost & Found application would not interfere with the existing e-commerce project.

Database setup completed

- Created a separate Atlas project.
- Created the lost-found-cluster.
- Used the Free cluster tier.
- Used Mumbai (ap-south-1) as the region.

- Created a separate database user.
- Connected the Node.js application to the new cluster.

A key debugging lesson occurred here: the first connection attempt used credentials from the existing e-commerce setup and produced bad auth: authentication failed. A separate Lost & Found database user was then created. The connection succeeded.

15. Environment Variables with .env

The MongoDB connection string was kept in server/.env rather than hard-coded into server.js.

```
MONGO_URI=mongodb+srv://USERNAME:PASSWORD@HOST/lost_found?...
```

The actual username, password and connection string are intentionally not reproduced here. A connection string contains credentials and should be treated as secret.

Loading the environment variable

```
const mongoose = require("mongoose")
require("dotenv").config()

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch((error) => console.log(error))
```

process.env.MONGO_URI reads the value stored in .env. This keeps credentials outside the main source code.

16. Mongoose Item Model

A models folder was created with Item.js. The schema defines the structure expected for Lost & Found items.

```
const mongoose = require("mongoose")

const itemSchema = new mongoose.Schema({
  name: String,
  category: String,
  description: String,
  date: String,
  location: String,
  image: String,
  status: String
})

module.exports = mongoose.model("Item", itemSchema)
```

A schema is a blueprint. The model created from it gives the server methods such as find() and save().

17. Replacing Hard-Coded GET Data with MongoDB

```
const Item = require("../models/Item")

app.get("/api/items", async (req, res) => {
  const items = await Item.find()
  res.json(items)
})
```

The keyword async allows the route to wait for the database operation. await Item.find() retrieves documents from MongoDB. Initially the result was [], which correctly meant the new database contained no items yet.

18. POST API for Creating Items

```
app.post("/api/items", async (req, res) => {
  const item = new Item(req.body)
  const savedItem = await item.save()
  res.json(savedItem)
})
```

This endpoint receives JSON from the frontend, creates a Mongoose Item from req.body, saves it to MongoDB, and returns the saved document.

19. Report Lost Form

A ReportLost.jsx page was first created as a simple placeholder, then turned into a proper form containing item name, category, description, date, location, optional image, and a submit button.

React form state

```
const [form, setForm] = useState({
  name: "",
  category: "Bags",
  description: "",
  date: "",
  location: ""
})

const handleChange = (e) => {
```

```

    setForm({
      ...form,
      [e.target.name]: e.target.value
    })
  }
}

```

...form copies the existing fields. [e.target.name] selects the field being changed, and e.target.value supplies its new value.

20. Connecting Report Lost to the POST API

```

const handleSubmit = (e) => {
  e.preventDefault()

  fetch("http://localhost:5000/api/items", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      ...form,
      type: "Lost",
      status: "Lost"
    })
  })
  .then((response) => response.json())
  .then((data) => {
    console.log(data)
    alert("Lost item reported successfully!")
  })
}

```

e.preventDefault() stops the browser's default form submission. fetch() sends the HTTP request. method: POST tells the server that new data is being submitted. JSON.stringify() converts the JavaScript object into JSON text for the request body.

Successful end-to-end test

A real iPhone 13 lost-item record was submitted from the Report Lost form. The browser endpoint http://localhost:5000/api/items then returned a MongoDB document containing the generated _id, name, category, description, date, location and status. This proved that the complete data flow worked.

```

React form
  ↓
fetch(POST /api/items)
  ↓
Express
  ↓
Mongoose Item
  ↓
MongoDB Atlas
  ↓
saved document

```

21. What Was Learned on Day 1

React

- Components and reusable UI.
- Props and case-sensitive prop names.
- `useState` and controlled inputs.
- Array filter/map concepts.
- React Router and URL-based pages.
- Basic CSS and Flexbox debugging.

Backend

- What an API endpoint is.
- GET versus POST.
- Express middleware such as `express.json()`.
- Request body via `req.body`.
- `async/await` for database operations.

MongoDB / Mongoose

- Atlas project, cluster and database user.
- Connection strings and environment variables.
- Mongoose schema and model.
- `find()` to retrieve documents.
- `save()` to insert a document.

22. Debugging Lessons

- Windows path issue: `'&'` in a folder name caused PowerShell trouble; renaming the folder solved it.
- CSS issue: DevTools helped distinguish a class/style problem from a React rendering problem.
- Props issue: `Type` and `type` are different property names.
- MongoDB auth issue: `'bad auth'` meant the application reached Atlas but the credentials were rejected.
- Empty array: `[]` from `GET /api/items` was correct before the first document was inserted.

23. Day 1 Final Checklist

✓	React/Vite frontend created
✓	Navbar and Home page
✓	Reusable ItemCard

✓	Browse page with search/type/category filters
✓	React Router pages
✓	Express server on port 5000
✓	GET /api/items
✓	MongoDB Atlas cluster and user
✓	Mongoose connection
✓	Item schema/model
✓	POST /api/items
✓	Report Lost form
✓	Real item successfully stored in MongoDB

24. Quick Mental Model for Revision

```

USER TYPES
  ↓
React state
  ↓
React form
  ↓
fetch()
  ↓
Express route
  ↓
Mongoose model
  ↓
MongoDB
  ↓
response JSON
  ↓
React UI

```

If you can explain this flow in your own words, you understand the core of what was built on Day 1.

25. What Was Not Finished Yet

- Report Found should use the same backend pattern.
- Browse should be fully driven by the MongoDB GET endpoint rather than only local sample data.
- Authentication/login/signup still need implementation.
- Item details, status updates and claiming still need implementation.
- Admin dashboard and approval/rejection still need implementation.
- Matching logic still needs implementation.
- Image upload is currently only a visual file input; actual file storage/upload is not implemented.

These are future project steps. Day 1 intentionally focused on getting the full foundation working rather than trying to complete every feature at once.

End of Day 1

The important milestone is not the amount of code. It is that the application now has a working path from a React form to a real MongoDB database through an Express backend.